PROYECTO EVALUABLE — ACCESO A DATOS / PROGRAMACIÓN

Hospital Management System

Memoria del Proyecto

Asignatura Acceso a Datos / Programación
Curso 2025 – 2026
Centro I.E.S. Delgado Hernández
Localidad Bollullos Par del Condado
Fecha entrega 27 / 04 / 2026

Equipo
— Héctor Rodríguez
— Javier Fernández
— Carlos Fernández
— Rafa Navarro
— Antonio Manuel Rodríguez

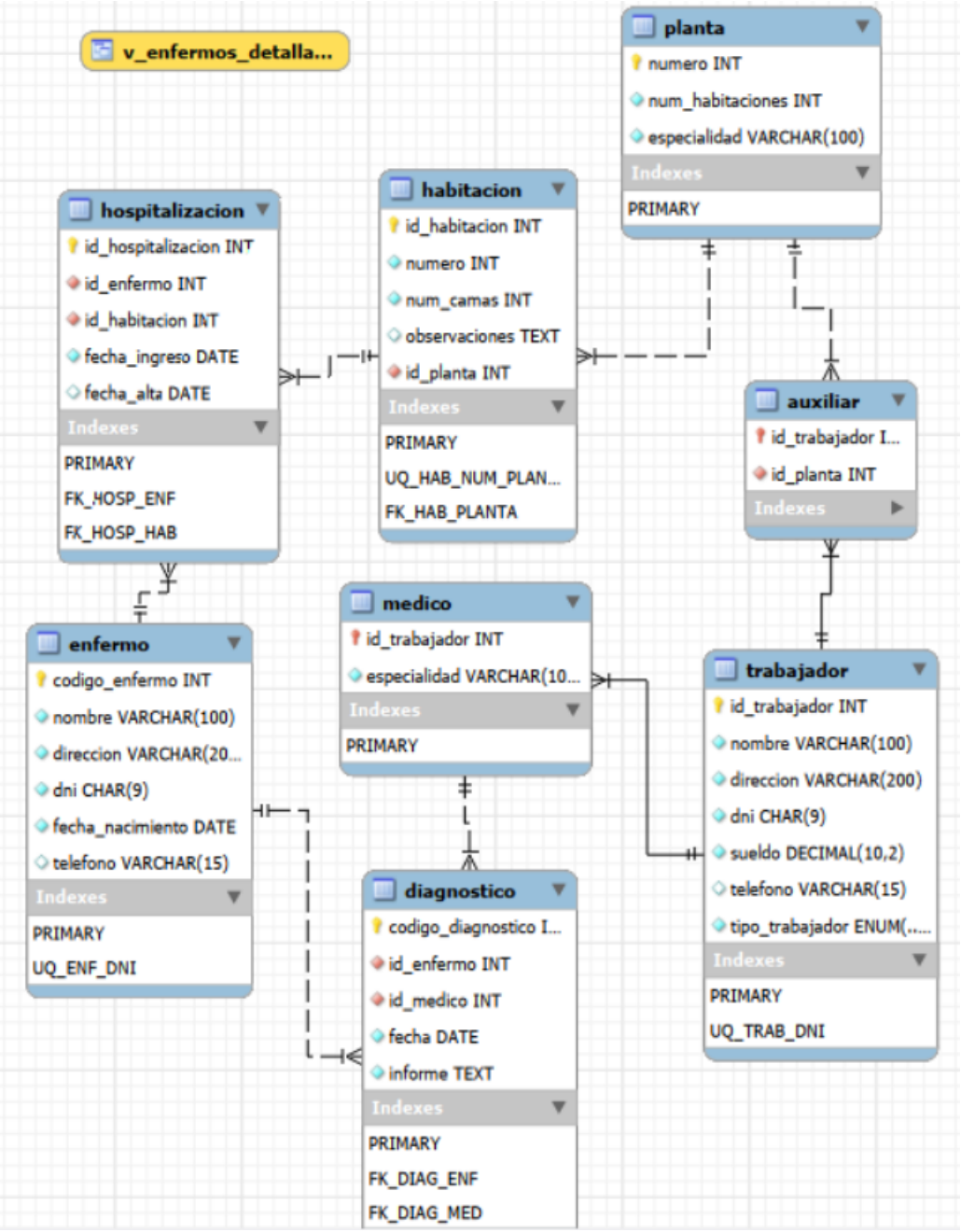
01 DISTRIBUCIÓN DE ROLES

Tabla de roles, tareas concretas y porcentaje de dedicación de cada miembro del equipo.

Miembro	Rol asignado	Tareas principales	Dedicación
Héctor Rodríguez	Diseñador de Base de Datos	Elabora el modelo conceptual (diagrama E-R), el modelo lógico y el diseño de las tablas. Define restricciones de integridad, PKs y FKs. Añadió archivos BD, data.sql y la normalización.	~20 %
Javier Fernández	Jefe de Proyecto	Coordina el equipo, gestiona plazos, organiza reuniones y supervisa objetivos. Realizó el commit inicial, añadió dependencias y la configuración Maven. Revisó y aprobó todos los Pull Requests.	~20 %
Carlos Fernández	Desarrollador Backend	Implementa la lógica de negocio en Java: clases de modelo (Trabajador, Medico, Auxiliar, Enfermo, Planta, Habitación, Hospitalización, Diagnostico), interfaces DAO y sus implementaciones JDBC para SQLite.	~20 %
Rafa Navarro	Calidad y Documentación	Redacta la memoria del proyecto, elabora diagramas de clases, revisa calidad del código, añade explicaciones de cada paquete, mantiene el README y el repositorio actualizado.	~20 %
Antonio M. Rodríguez	Desarrollador de Interfaz	Diseña e implementa la interfaz de usuario por consola: clase Main, todos los submenús CRUD (MenuEnfermos, MenuMedicos, MenuAuxiliares, MenuPlantas, MenuHabitaciones, MenuHospitalizaciones, MenuDiagnosticos) y la clase Utilidades.	~20 %

02 DIAGRAMA ENTIDAD-RELACIÓN

Modelo conceptual completo: entidades, atributos, relaciones y cardinalidades.



2.1 Entidades y atributos

Entidad	Atributos relevantes	Observaciones de diseño
Enfermo	idEnfermo (PK), nombre, dirección, DNI (UNIQUE), teléfono, fechaNacimiento	La edad no se almacena; se calcula dinámicamente en Java con Period.between().

Entidad	Atributos relevantes	Observaciones de diseño
Trabajador (abstracta)	idTrabajador (PK), nombre, dirección, DNI (UNIQUE), sueldo, teléfono	Clase padre abstracta en Java. En BD, Médico y Auxiliar tienen tablas independientes con todos los atributos.
Médico	Hereda de Trabajador + especialidad	Tabla propia (médicos). Añade el atributo especialidad al conjunto de atributos de Trabajador.
Auxiliar	Hereda de Trabajador + idPlanta (FK)	Tabla propia (auxiliares). Cada auxiliar está asignado a una única planta mediante FK.
Planta	numeroPlanta (PK), cantidadHabitaciones, especialidad	La numeración de planta es única. Una planta puede tener muchas habitaciones y auxiliares.
Habitación	numeroHabitacion (PK), numeroPlanta (FK), cantidadCamas, observaciones	Cada habitación pertenece a una planta. En una habitación puede haber más de un enfermo hospitalizado.
Hospitalización	idHospitalizacion (PK), idEnfermo (FK), numeroHabitacion (FK), numeroPlanta, fechaIngreso, fechaAlta	Registra el ingreso de un enfermo. La fecha de alta es NULL mientras el enfermo sigue hospitalizado.
Diagnóstico	codigo (PK), dniMedico (FK), idEnfermo (FK), fecha, informe	Cada diagnóstico es emitido por un único médico. Se referencia al médico mediante su DNI.

2.2 Decisiones de diseño

Jerarquía Médico / Auxiliar

Se optó por una estrategia de herencia por tabla concreta (Table Per Concrete Class). En lugar de crear una única tabla TRABAJADOR con columnas opcionales, se crearon tablas independientes para médicos y auxiliares, cada una con todos los atributos propios de un trabajador más los específicos de su tipo. En Java, esta jerarquía se modela con la clase abstracta Trabajador y las subclases Medico y Auxiliar. Esta decisión evita columnas nulas y simplifica las consultas JDBC, a costa de cierta duplicación de columnas comunes.

Relación Habitación ↔ Planta

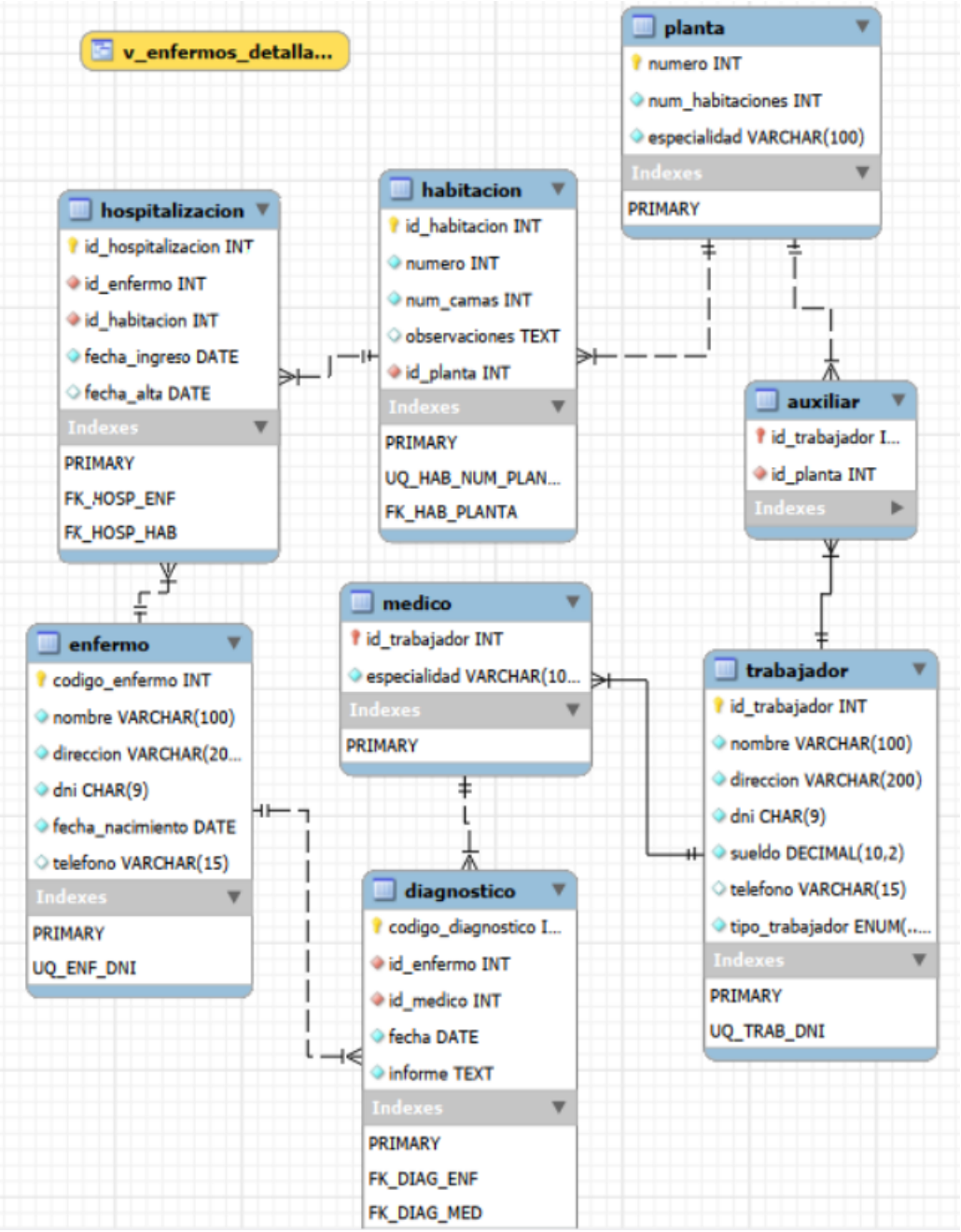
Se implementó como una relación N:1: muchas habitaciones pertenecen a una planta. La tabla habitaciones incluye la columna numeroPlanta como clave foránea hacia plantas. La unicidad de número de habitación dentro de una misma planta queda garantizada por la PK numeroHabitacion en el diseño actual. En el script schema.sql de referencia se añade además un índice UNIQUE(numero, id_planta).

Asignación de diagnósticos a enfermos

La tabla diagnósticos registra el DNI del médico como referencia (columna dniMedico) y el idEnfermo como FK. Esto permite consultar el historial completo de diagnósticos de cualquier enfermo o filtrar por médico, cumpliendo con el requisito de que cada diagnóstico sea emitido por un único médico.

03 DIAGRAMA DE TABLAS DE LA BASE DE DATOS

Esquema relacional SQLite definitivo: tablas creadas automáticamente al arrancar la aplicación mediante los métodos crearTabla() de cada DAO.



3.1 Esquema de tablas

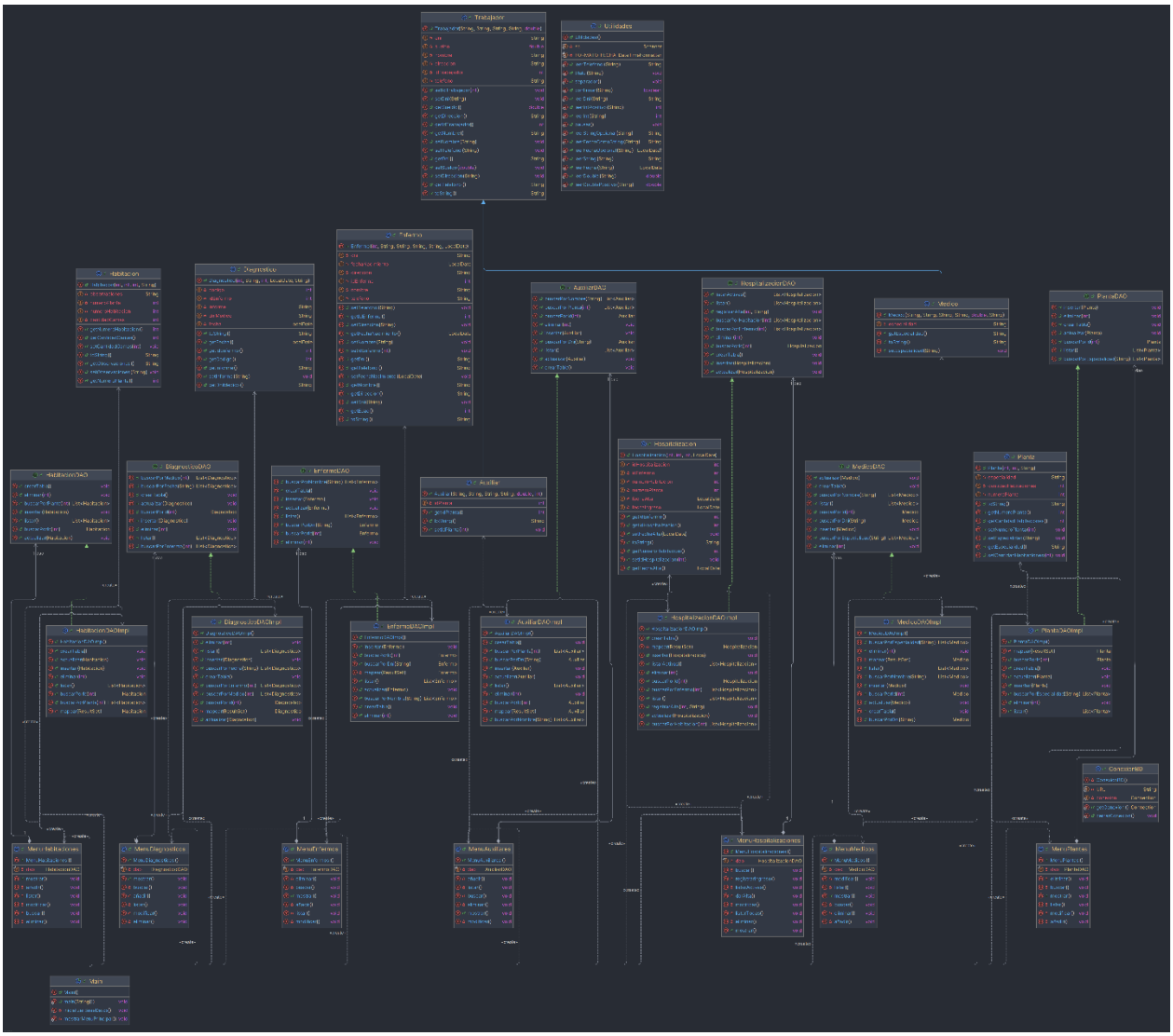
Tabla	Columna	Tipo	Clave	Restricciones
enfermos	idEnfermo	INTEGER	PK	AUTOINCREMENT, NOT NULL

Tabla	Columna	Tipo	Clave	Restricciones
	nombre	TEXT		NOT NULL
	dirección	TEXT		—
	DNI	TEXT		NOT NULL, UNIQUE
	teléfono	TEXT		—
	fechaNacimiento	TEXT		NOT NULL (ISO-8601)
plantas	numeroPlanta	INTEGER	**PK**	NOT NULL
	cantidadHabitaciones	INTEGER		NOT NULL
	especialidad	TEXT		NOT NULL
médicos	id	INTEGER	**PK**	AUTOINCREMENT, NOT NULL
	nombre, dirección, teléfono	TEXT		nombre NOT NULL
	DNI	TEXT		NOT NULL, UNIQUE
	sueldo	REAL		NOT NULL
	especialidad	TEXT		NOT NULL
auxiliares	id	INTEGER	**PK**	AUTOINCREMENT, NOT NULL
	nombre, dirección, teléfono	TEXT		nombre NOT NULL
	DNI	TEXT		NOT NULL, UNIQUE
	sueldo	REAL		NOT NULL
	idPlanta	INTEGER	**FK**	FK → plantas(numeroPlanta)
habitaciones	numeroHabitacion	INTEGER	**PK**	NOT NULL
	numeroPlanta	INTEGER	**FK**	FK → plantas(numeroPlanta)
	cantidadCamas	INTEGER		NOT NULL
	observaciones	TEXT		—
hospitalizaciones	idHospitalizacion	INTEGER	**PK**	AUTOINCREMENT
	idEnfermo	INTEGER	**FK**	FK → enfermos(idEnfermo)
	numeroHabitacion	INTEGER	**FK**	FK → habitaciones(numeroHabitacion)
	numeroPlanta	INTEGER		NOT NULL
	fechaIngreso	TEXT		NOT NULL (ISO-8601)
	fechaAlta	TEXT		NULL si sigue hospitalizado
diagnósticos	codigo	INTEGER	**PK**	AUTOINCREMENT
	dniMedico	TEXT		NOT NULL
	idEnfermo	INTEGER	**FK**	FK → enfermos(idEnfermo)
	fecha	TEXT		NOT NULL (ISO-8601)
	informe	TEXT		—

Nota: Las fechas se almacenan como TEXT en formato ISO-8601 (yyyy-MM-dd). Java las gestiona con LocalDate y toString() / LocalDate.parse().

04 DIAGRAMA DE CLASES UML

Diagrama UML con las clases del proyecto Java: modelo, DAO (interfaces e implementaciones), conexión e interfaz de consola.



4.1 Capa de Modelo (paquete modelo)

Clase	Tipo	Atributos principales	Responsabilidad
Trabajador	Abstracta	nombre, dirección, teléfono, DNI, sueldo, idTrabajador	Clase padre de la jerarquía de trabajadores. Define atributos y getters/setters comunes. Incluye toString().
Medico	extends Trabajador	especialidad	Representa a un médico. Hereda de Trabajador y añade el atributo especialidad.
Auxiliar	extends Trabajador	idPlanta	Representa a un auxiliar de enfermería. Hereda de

Clase	Tipo	Atributos principales	Responsabilidad
			Trabajador y añade la referencia a su planta asignada.
Enfermo	Concreta	idEnfermo, nombre, dirección, DNI, teléfono, fechaNacimiento	Representa a un paciente. Incluye el método getEdad() que calcula la edad actual a partir de fechaNacimiento.
Planta	Concreta	numeroPlanta, cantidadHabitaciones, especialidad	Representa una planta del hospital con su especialidad médica.
Habitación	Concreta	numeroHabitacion, numeroPlanta, cantidadCamas, observaciones	Representa una habitación física. Se asocia a una planta mediante numeroPlanta.
Hospitalización	Concreta	idHospitalizacion, idEnfermo, numeroHabitacion, numeroPlanta, fechaIngreso, fechaAlta	Registra la estancia de un enfermo. fechaAlta es null mientras el paciente sigue ingresado.
Diagnostico	Concreta	codigo, dniMedico, idEnfermo, fecha, informe	Representa un diagnóstico clínico. Se referencia al médico por su DNI y al enfermo por su ID.

4.2 Capa de Acceso a Datos (paquetes dao y dao.impl)

Interface / Clase	Capa	Métodos principales
[Entidad]DAO (×7)	Interfaz DAO	crearTabla(), insertar(), listar(), buscarPorId(), buscarPorDni(), buscarPorNombre(), actualizar(), eliminar(). Cada entidad tiene su interfaz específica.
[Entidad]DAOImpl (×7)	Implementación JDBC	Implementa la interfaz DAO correspondiente usando PreparedStatement y ConexionBD.getConexion(). Incluye mapear(ResultSet) para construir objetos de modelo desde los resultados SQL.

4.3 Capas de Conexión e Interfaz (paquetes conexión y app)

Clase	Paquete	Responsabilidad
ConexionBD	conexion	Patrón Singleton. Mantiene una única conexión JDBC a hospital.db (SQLite). Métodos: getConexion() y cerrarConexion().
Main	app	Punto de entrada. Inicializa todas las tablas al arrancar, muestra el menú principal e instancia los submenús. Gestiona el cierre de conexión al salir.
Utilidades	app	Métodos estáticos de apoyo: leerInt(), leerString(), titulo(), separador(). Centraliza la lectura de entrada del usuario.
Menu[Entidad] (×7)	app.Menus	Cada clase gestiona el submenú CRUD de su entidad (listar, buscar, añadir, modificar, eliminar). Instancia su DAOImpl y delega en él las operaciones de BD.

05 JUSTIFICACIÓN DE TECNOLOGÍAS

Este apartado recoge la experiencia real del equipo con cada tecnología: por qué se eligió, qué ventajas aportó y qué dificultades se encontraron.

5.1 Lenguaje: Java 22

Justificación

Java es el lenguaje principal del módulo y con el que el equipo tiene mayor experiencia práctica. Su orientación a objetos encaja directamente con el modelo del dominio: la jerarquía Trabajador → Médico/Auxiliar se implementa de forma natural con clases abstractas y herencia. Además, la API JDBC está integrada en el JDK, lo que evita dependencias adicionales para la comunicación con la base de datos.

Ventajas experimentadas

El tipado estático detectó numerosos errores en tiempo de compilación antes de ejecutar la aplicación, lo que aceleró el ciclo de depuración. Las clases de java.time (LocalDate, Period) simplificaron el manejo de fechas y el cálculo de la edad de los pacientes sin librerías externas. Los bloques try-with-resources garantizaron el cierre automático de conexiones y statements, reduciendo fugas de recursos.

Dificultades encontradas

Al principio se produjeron errores al no cerrar correctamente los ResultSet y PreparedStatement, lo que causaba bloqueos en SQLite. Se resolvió sistematizando el uso de try-with-resources en todos los DAOImpl. También fue necesario convertir manualmente las fechas entre String (SQLite) y LocalDate (Java) mediante toString() y LocalDate.parse().

5.2 Base de datos: SQLite

SGBD elegido: SQLite 3 — frente a la alternativa PostgreSQL.

Justificación de la elección

Se eligió SQLite porque no requiere instalar ni configurar un servidor de base de datos separado. La base de datos completa es un único archivo (hospital.db) que se genera automáticamente al arrancar la aplicación, lo que facilitó enormemente la puesta en marcha del proyecto en los distintos equipos del equipo de desarrollo sin dependencias de entorno. Esto fue especialmente útil al trabajar en máquinas distintas y compartir el proyecto por Git.

Comparativa SQLite vs PostgreSQL

Criterio	SQLite	PostgreSQL
Instalación	Ninguna (archivo .db)	Servidor + usuario + contraseña
Portabilidad	Muy alta: un solo archivo	Requiere configurar entorno
Rendimiento	Óptimo para volumen pequeño/medio	Superior en entornos concurrentes
Concurrencia	Limitada (escritura secuencial)	Alta (MVCC)
Adecuación al proyecto	Perfecta para aplicación de escritorio	Sobredimensionada para este uso

Ventajas experimentadas

La base de datos se creó y distribuyó sin ningún paso de configuración adicional. Al incluir el archivo hospital.db en el directorio raíz, cualquier miembro del equipo podía ejecutar la aplicación inmediatamente tras clonar el repositorio.

Dificultades encontradas

SQLite no soporta el tipo DATE nativo ni la instrucción ENGINE=InnoDB del script schema.sql (diseñado originalmente para MySQL). Se adaptaron todas las fechas a TEXT en formato ISO-8601 y se reescribieron las sentencias CREATE TABLE en cada DAOImpl con sintaxis compatible con SQLite. Además, la aplicación de FKs requirió activar PRAGMA foreign_keys = ON manualmente.

5.3 Conector JDBC: sqlite-jdbc 3.45.1.0

Justificación

Se usó el driver sqlite-jdbc de Xerial (org.xerial:sqlite-jdbc:3.45.1.0), declarado como dependencia Maven en el pom.xml. Es el driver oficial de referencia para SQLite en Java, mantiene compatibilidad con la API JDBC estándar y no requiere instalación de bibliotecas nativas externas, ya que las incluye en el propio JAR.

Ventajas experimentadas

Al respetar la API JDBC estándar (DriverManager, Connection, PreparedStatement, ResultSet), todo el código de acceso a datos es portátil: si en el futuro se decidiese migrar a PostgreSQL, bastaría con cambiar la URL de conexión y el driver sin tocar los DAOImpl. El uso de PreparedStatement evitó inyección SQL en todas las operaciones de escritura.

Patrón Singleton en ConexionBD

La clase ConexionBD implementa el patrón Singleton: mantiene una única instancia de Connection estática. getConexion() comprueba si la conexión es nula o está cerrada y solo crea una nueva en ese caso, evitando la sobrecarga de abrir múltiples conexiones por cada operación DAO.

5.4 IDE: IntelliJ IDEA

Justificación

El equipo utiliza IntelliJ IDEA en clase diariamente, por lo que era el entorno en el que todos tenían mayor dominio. Su integración con Maven, el soporte de autocompletado avanzado, la refactorización automática y el debugger visual aceleraron significativamente el desarrollo. La herramienta de Database (pestaña de base de datos) permitió inspeccionar hospital.db directamente desde el IDE.

5.5 Herramientas adicionales

Herramienta	Uso en el proyecto	Justificación
Apache Maven	Gestión de dependencias y compilación	Permite declarar sqlite-jdbc como dependencia en pom.xml y compilar con mvn package sin configuración manual del classpath.
MySQL Workbench	Modelado E-R y generación de schema.sql	Héctor utilizó Workbench para diseñar el diagrama relacional (DiagramaHospital.mwb) y exportar el script SQL de referencia.
Git + GitHub	Control de versiones y colaboración	Repositorio público en GitHub con estrategia de ramas por rol. Gestión de Pull Requests para integrar cambios en main.
SourceTree	Visualización del historial de Git	Permitió a los miembros del equipo revisar el grafo de ramas y gestionar merges de forma visual sin necesidad de memorizar comandos Git.

06 ESTRATEGIA DE GIT

6.1 Modelo de ramas

Se adoptó una estrategia basada en ramas por rol, inspirada en GitHub Flow. Cada miembro trabajó en su rama temática y la integración en main se realizó exclusivamente mediante Pull Requests revisados por el Jefe de Proyecto (Javier Fernández). La rama main siempre contiene una versión funcional del proyecto.

Rama	Propósito	Responsable / criterio de merge
main	Versión definitiva y siempre funcional del proyecto. Solo recibe cambios por Pull Request aprobado.	Javier Fernández (Jefe de Proyecto). Revisión y aprobación obligatoria del PR.
Backend	Desarrollo de clases de modelo, interfaces DAO e implementaciones JDBC.	Carlos Fernández. Merge en main vía PR #2 el 28/04/2026.
BaseDeDatos	Diseño del diagrama E-R, script schema.sql, data.sql y normalización de la BD.	Héctor Rodríguez. Merge en main vía PR #4 el 28/04/2026.
InterfazGrafica	Implementación de todos los menús de consola, clase Main y Utilidades.	Antonio Manuel Rodríguez. Merge en main vía PR #3 el 28/04/2026.
documentacion	README.md, explicaciones de packages, memoria del proyecto.	Rafa Navarro. Rama activa durante todo el desarrollo.

6.2 Convención de commits

El equipo adoptó mensajes de commit descriptivos en español, sin prefijos formales tipo Conventional Commits, pero siguiendo la convención de describir con claridad qué cambio se realizó. Ejemplos reales extraídos del historial:

Commit real	Autor
Commit inicial	Javier Fernández
Creación clase abstracta Trabajador	Carlos Fernández
Creaciones clases Médico y Auxiliar + javadoc de esas clases	Carlos Fernández
Añadir archivos base de datos	Héctor Rodríguez
Creación de interfaces	Javier Fernández
Creación implementaciones DAO	Carlos Fernández
feat: implementar interfaz de usuario con menús CRUD por consola	Antonio M. Rodríguez
añadir documentacion con formato word	Rafa Navarro
Corrección errores (utilidades y menus) — commit más reciente en main	Javier Fernández

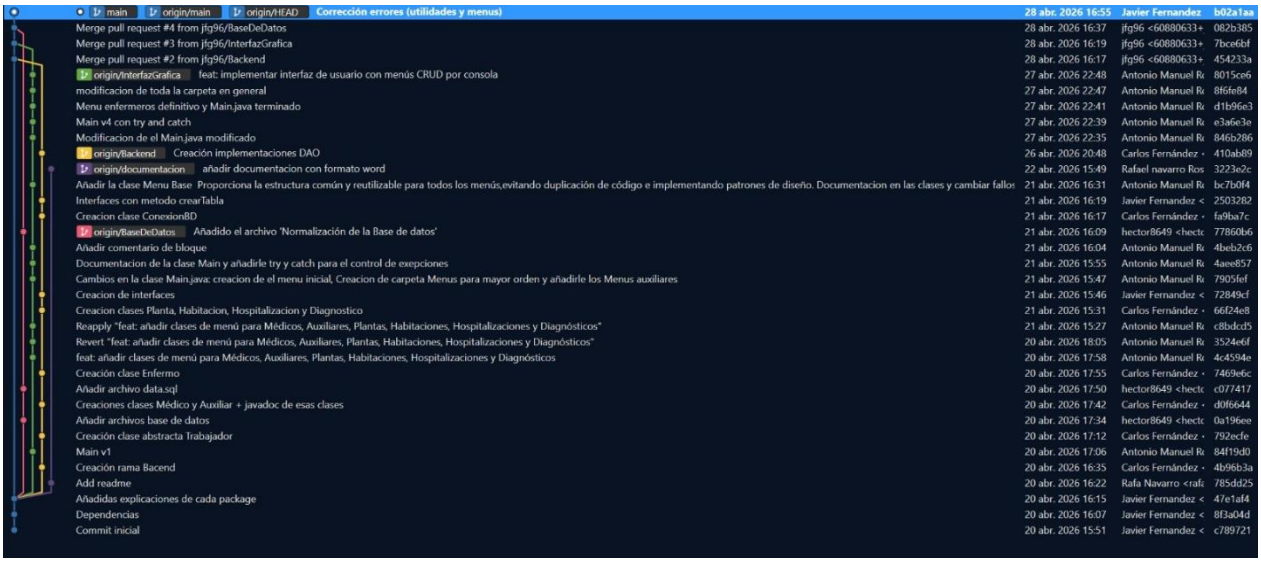
6.3 Pull Requests y revisiones

Se realizaron 4 Pull Requests a lo largo del desarrollo, todos revisados y aprobados por el Jefe de Proyecto antes de fusionarse en main:

PR	Rama origen	Fecha	Aprobador	Descripción
#1	documentacion	20/04/2026	Javier Fernández	Add readme (merge directo por ser el primer contenido de rama)
#2	Backend	28/04/2026	Javier Fernández	Integración de modelos Java, interfaces DAO y sus implementaciones JDBC
#3	InterfazGrafica	28/04/2026	Javier Fernández	Integración de todos los menús de consola y Main.java
#4	BaseDeDatos	28/04/2026	Javier Fernández	Integración de archivos de BD: schema.sql, data.sql y normalización

No se registraron conflictos de merge graves, ya que cada miembro trabajó sobre ficheros claramente separados por paquete. El único ajuste necesario fue en Main.java, que era compartido entre InterfazGrafica y Backend; se resolvió coordinando el merge entre Javier y Antonio Manuel.

6.4 Evidencia del historial



07 ESTRUCTURA DEL PROYECTO

Árbol de directorios real del repositorio hospital, con descripción de cada carpeta y archivo relevante.

```
hospital/
├── pom.xml                        ← Configuración Maven (Java 22, sqlite-jdbc)
├── .gitignore                     ← Excluye target/, *.class, *.db local
├── identifier.sqlite              ← BD SQLite generada en ejecución
└── src/main/java/
    ├── app/
    │   ├── Main.java              ← Punto de entrada + menú principal
    │   ├── Utilidades.java        ← Métodos de E/S de consola compartidos
    │   └── Menus/
    │       ├── MenuEnfermos.java
    │       ├── MenuMedicos.java
    │       ├── MenuAuxiliares.java
    │       ├── MenuPlantas.java
    │       ├── MenuHabitaciones.java
    │       ├── MenuHospitalizaciones.java
    │       ├── MenuDiagnosticos.java
    │       └── Utilidades.java    ← Utilidades específicas de menús
    ├── conexion/
    │   └── ConexionBD.java        ← Singleton JDBC
    ├── dao/
    │   ├── [Entidad]DAO.java ×7   ← Interfaces CRUD
    │   └── impl/
    │       └── [Entidad]DAOImpl.java ×7 ← Implementaciones SQLite
    ├── modelo/
    │   ├── Trabajador.java        ← Clase abstracta
    │   ├── Medico.java / Auxiliar.java
    │   ├── Enfermo.java / Planta.java
    │   ├── Habitacion.java / Hospitalizacion.java
    │   └── Diagnostico.java
    └── resources/
        ├── Diagrama E-R.png       ← Diagrama conceptual
        ├── DiagramaHospital.mwb   ← Modelo MySQL Workbench
        ├── schema.sql             ← Script DDL de referencia (MySQL)
        └── data.sql               ← Datos de prueba iniciales
```

7.1 Descripción de paquetes

Paquete / Archivo	Contenido y responsabilidad
app	Capa de presentación: punto de entrada (Main), lógica de menú y clase de utilidades de consola. Antonio Manuel fue el responsable principal de este paquete.
app.Menus	Siete clases de submenú, una por entidad del sistema. Cada una ofrece un CRUD interactivo por consola, instancia su DAOImpl y gestiona la entrada del usuario.
conexion	Patrón Singleton para la conexión JDBC. ConexionBD centraliza la apertura y cierre de la conexión a hospital.db, evitando conexiones duplicadas.
dao	Siete interfaces DAO, una por entidad. Definen el contrato CRUD (crearTabla, insertar, listar, buscarPor*, actualizar, eliminar). Javier Fernández creó las interfaces iniciales.

Paquete / Archivo	Contenido y responsabilidad
dao.impl	Implementaciones concretas de cada interfaz DAO para SQLite. Usan PreparedStatement y ConexionBD. Desarrolladas por Carlos Fernández.
modelo	Clases POJO del dominio: Trabajador (abstracta), Medico, Auxiliar, Enfermo, Planta, Habitación, Hospitalización y Diagnostico. Desarrolladas por Carlos Fernández con javadoc completo.
resources	Recursos estáticos: diagrama E-R (PNG), modelo Workbench (MWB), script SQL de referencia (schema.sql) y datos de prueba (data.sql). Generados por Héctor Rodríguez.

08 REVISIÓN DE CALIDAD DEL CÓDIGO

Registro de revisiones realizadas por el responsable de calidad (Rafa Navarro) a lo largo del desarrollo.

8.1 Registro de revisiones

Fecha	Módulo revisado	Observaciones	Responsable	Estado
21/04/2026	modelo / dao	Verificado uso correcto de Javadoc en todas las clases de modelo y DAOImpl. Confirmado patrón try-with-resources en todos los métodos DAO.	Rafa Navarro	**OK**
22/04/2026	conexion	Revisado ConexionBD: patrón Singleton correcto, cierre de conexión gestionado en Main. Constructor privado para evitar instanciación.	Rafa Navarro	**OK**
27/04/2026	app.Menus	Revisados todos los submenús. Se detectó ausencia de validación de entradas nulas en algunos métodos; corregido en el commit 'Corrección errores (utilidades y menus)'.	Rafa Navarro	**OK**
28/04/2026	Integración main	Revisión final tras los 3 merges de PR. Verificado arranque completo de la aplicación, inicialización de tablas y operaciones CRUD básicas sobre todas las entidades.	Rafa Navarro	**OK**

8.2 Convenciones de código aplicadas

Convención	Aplicación en el proyecto
Javadoc	Todos los atributos, constructores y métodos de las clases de modelo llevan comentarios Javadoc con @param y @return. Las clases DAOImpl llevan comentario de clase.
Nombrado camelCase	Variables y métodos en camelCase (getIdEnfermo, buscarPorDni). Clases en PascalCase (EnfermoDAOImpl). Constantes en UPPER_SNAKE_CASE (URL en ConexionBD).
PreparedStatement	Toda consulta con parámetros usa PreparedStatement, nunca concatenación de strings, para prevenir inyección SQL.
try-with-resources	Todos los bloques que abren Connection, Statement o ResultSet usan try-with-resources para garantizar el cierre automático.
Separación por capas	Ninguna clase de Menus contiene SQL directo. Toda operación de BD pasa por el DAO correspondiente.

8.3 Deuda técnica identificada

Los siguientes aspectos quedaron fuera del alcance del proyecto por restricciones de tiempo, pero se han identificado como mejoras futuras:

Aspecto	Descripción
FK en SQLite	SQLite requiere PRAGMA foreign_keys = ON por sesión. Actualmente no se activa explícitamente en ConexionBD, lo que podría permitir inserciones inconsistentes.
Validación de entradas	La validación de DNI (formato, longitud) y fechas (coherencia) es básica. En una versión futura se añadirían expresiones regulares y comprobaciones de rango.
Manejo de excepciones	Los errores SQL se imprimen por stderr pero no se propagan al usuario con mensajes amigables en todos los casos.
Tests unitarios	No se han implementado tests automáticos (JUnit). Las pruebas se realizaron manualmente sobre la consola.
Interfaz gráfica	La aplicación opera completamente por consola. Una interfaz Swing o JavaFX mejoraría la experiencia de usuario.